Chuleta de examen
Git · GitHub · Actions

Despliegue de Aplicaciones Web · 2º DAW. Busca por color, copia la receta y comprueba siempre con **git status** y **git log --oneline --graph --all**.

0 · Recetas de examen
Lo que suele caer, paso a paso

1 · Terminal y editores
ls, cd, cat, nano, vi, heredoc

2 · Git local
init, add, commit, diff, log, gitignore

3 · Deshacer
reset, revert, amend, rm --cached

4 · Remotos y GitHub
token, remote, push, pull, fetch, clone

5 · Ramas y merge
branch, switch, merge, conflictos

6 · Tags
ligeros, anotados, subir, borrar

7 · GitHub Pages, fork y PR
gh-pages, upstream, buenas prácticas

8 · Gitflow y teoría SCV
ramas, comandos, tipos de SCV

9 · Actions: workflows
esqueleto, on, needs, if, env, matrix

10 · Práctica euro.py
las 5 tareas resueltas

11 · Custom Actions
action.yml, Docker, JavaScript

12 · Errores típicos
mensaje → solución

🎯 Las 5 reglas del examen

1. Antes de cada paso: `git status` y `git branch` (¿dónde estoy?).
2. Todo lo de Git empieza por `git` . `--soft` , `--hard` , `--staged` van **sin espacio**.
3. Para "hacer un cambio": `echo "algo" >> fichero` o nano. `touch` en un fichero que ya existe **no cambia nada**.
4. `git push` **no sube tags**. Súbelos aparte.
5. Al final: `git log --oneline --graph --all --decorate` para comprobar que todo está como piden.

⚡ Antes de empezar en la máquina

`git config --global user.name "Fabián Gorbaran"` · `git config --global user.email "tu@email"` · `git config --global core.editor nano` · `git config --global credential.helper store`

Copiar/pegar en la terminal: **Ctrl+Shift+C / Ctrl+Shift+V**. Ten el **token** de GitHub (permisos repo + workflow) apuntado.

0 Recetas de examen

copia, cambia nombres, comprueba

R1 · Crear repo y primer commit

```
mkdir -p ~/git/web && cd ~/git/web
git init
git branch --show-current      # si dice main y piden master:
git branch -m master
echo "# Mi web" > Readme.md
git add Readme.md
git commit -m "Primer commit: añade Readme"
```

R2 · Subir a GitHub repo creado VACÍO en la web

```
git remote add origin https://github.com/USUARIO/web.git
git push -u origin master          # Username: tu usuario · Password: el TOKEN
git push origin --tags             # si hay tags
```

R3 · Rama, commits, divergir y fusionar la práctica de branques i tags

```
git switch -c contacte           # crear y moverse
cat > index.html << 'EOF'
<!DOCTYPE html>
<html><body><h1>Mi web</h1></body></html>
EOF
git add index.html && git commit -m "Añade index base"
nano index.html                    # añade la sección
git commit -am "Añade sección de contacto"
nano index.html                    # añade el enlace
git commit -am "Añade enlace de contacto"
git log --oneline                  # "cuántos commits"
git switch master                  # sin fusionar todavía
echo '<meta name="description" content="...">' > new.html
git add new.html && git commit -m "Añade new.html"
git log --oneline --graph --all    # se ve que divergen
git merge contacte                 # NO es fast-forward → nano: Ctrl+O, Enter, Ctrl+X
git log --oneline --graph --all
git switch contacte && ls        # sin new.html
git switch master && ls          # con todo
```

R4 · Tag de entrega, limpiar y entregar

```
git switch master
git tag -a entrega-fabian -m "Entrega prueba ramas y tags - Fabián"
git show entrega-fabian
git branch -d contacte             # borrar la rama ya fusionada
git remote add origin https://github.com/USUARIO/web.git
git push -u origin master
git push origin entrega-fabian       # el tag aparte
git log --oneline --graph --all --decorate
```

R5 · Resolver un conflicto

```
git merge rama                    # CONFLICT (content): ...
git status                        # "both modified: index.html"
nano index.html                    # deja lo bueno y BORRA <<<<<<<  =======  >>>>>>>
git add index.html
git commit -m "Resuelve conflicto"
# o cancelar todo:  git merge --abort
```

R6 · Workflow nuevo en un repo

```
mkdir -p .github/workflows
nano .github/workflows/ci.yml        # esqueleto de la sección 9
git add .github && git commit -m "Añade workflow" && git push
# GitHub → pestaña Actions → ejecución → job → despliega el step
```

R7 · Clonar el repo del profe y subirlo al tuyo

```
git clone https://github.com/titansmc/actions.git && cd actions
git remote set-url origin https://github.com/USUARIO/actions.git   # set-url, NO add
git push -u origin main
```

1 Terminal y editores

Comando	Hace
`pwd`	dónde estoy
`ls -la`	listar todo, también ocultos
`ls -R`	listar con subcarpetas
`cd dir` · `cd ..` · `cd`	entrar · subir · ir a home
`mkdir -p a/b/c`	crear carpetas de golpe
`touch f`	crear fichero vacío
`cat f`	ver contenido
`echo "x" > f`	escribir (machaca)
`echo "x" >> f`	añadir al final
`cp a b` · `mv a b`	copiar · mover/renombrar
`rm f` · `rm -r dir`	borrar fichero · carpeta
`source ~/.bashrc`	aplicar cambios del bashrc
`chmod +x f.sh`	hacer ejecutable

nano	
Ctrl+O Enter	guardar
Ctrl+X	salir
Ctrl+K / Ctrl+U	cortar / pegar línea
Ctrl+W	buscar

vi / vim	
i	escribir
Esc	dejar de escribir
:wq Enter	guardar y salir
:q! Enter	salir sin guardar
dd · **u**	borrar línea · deshacer

Símbolo	
`~`	home
`.` · `..`	actual · arriba

Escribir varias líneas de golpe (heredoc)

```
cat > index.html << 'EOF'
<h1>Hola</h1>
EOF
```

> ⚡ **Atajos**
> **Tab** autocompleta · ↑ comando anterior · **Ctrl+C** cancelar · **q** salir de `git log` · **clear** limpiar.

2 Git local

Las 3 zonas

Zona	Estado	Llegas con
Working	modificado / untracked	editar
Staging	preparado	`git add`
Repo (.git)	confirmado	`git commit`

Comando	Hace
`git init`	crear repo (.git)
`git init --bare`	repo "servidor"
`git status`	estado
`git add f`	preparar uno
`git add .`	preparar todo
`git add -u`	solo los ya seguidos
`git add -n .`	simular
`git commit -m "M"`	confirmar
`git commit -am "M"`	add+commit (NO ficheros nuevos)
`git commit -m "Asunto" -m "Cuerpo"`	asunto + cuerpo
`git mv a b`	renombrar
`git rm f`	borrar y preparar

Ver cambios e historial

Comando	Hace
`git diff`	working vs staging
`git diff --staged`	staging vs último commit
`git diff r1 r2`	entre ramas
`git log`	historial completo
`git log --oneline`	una línea por commit
`git log -3`	los 3 últimos
`git log -p`	con diferencias
`git log --oneline --graph --all`	dibujo de ramas
`git shortlog`	agrupado por autor
`git shortlog -sn`	nº commits por autor
`git show HASH`	detalle de un commit
`git show --stat HEAD`	ficheros del último commit

.gitignore

```
*.log            # todos los .log
!importante.log  # excepción (DEBAJO)
/TODO            # solo el de la raíz
build/           # carpeta entera
doc/*.txt        # .txt de doc
.env             # secretos
venv/            # entorno Python
```

⚠️ .gitignore

Solo afecta a ficheros que Git **aún no sigue**. Si ya hiciste commit: `git rm --cached f` + añadirlo al .gitignore + commit.

🎯 Qué se ignora

contraseñas/API keys, `.vscode`, logs, ficheros del sistema, `dist` / `build`, `node_modules`, coverage.

🎯 Buen mensaje de commit

Asunto ≤ **50** caracteres, **mayúscula**, **sin punto**, **imperativo** ("Añade", no "Añadido") · línea en blanco · cuerpo ≤ **72** por línea explicando qué y por qué.

3 · Deshacer

Quiero…	Comando
Sacar un fichero de staging	`git reset f` · `git restore --staged f`
Descartar cambios de un fichero (volver al último commit)	`git checkout -- f` · `git restore f`
Cambiar mensaje del último commit	`git commit --amend -m "Nuevo"`
Meter un fichero olvidado en el último commit	`git add f` + `git commit --amend --no-edit`
Quitar último commit, cambios siguen **preparados**	`git reset --soft HEAD~1`
Quitar último commit, cambios en **working**	`git reset HEAD~1` (mixed, por defecto)
Quitar último commit y **BORRAR** cambios	`git reset --hard HEAD~1` ⚠️
Deshacer un commit creando otro (seguro si ya está subido)	`git revert HASH` · `git revert HEAD`
Dejar de seguir un fichero sin borrarlo del disco	`git rm --cached f`

	reset	revert
¿Borra historial?	Sí	No
¿Commit nuevo?	No	Sí
¿Seguro en equipo?	❌	✅

💡 **HEAD**
`HEAD` = último commit de tu rama · `HEAD~1` = el anterior · `HEAD~2` = dos atrás.

⚠️
`--amend` y `reset` solo con commits **no subidos**.

4 Remotos y GitHub

Comando	Hace
`git clone URL [carpeta]`	descargar repo entero (crea `origin`)
`git remote -v`	ver remotos
`git remote add origin URL`	añadir remoto
`git remote set-url origin URL`	cambiar URL (si ya existe)
`git remote show origin`	info detallada
`git remote remove origin`	quitar remoto
`git push -u origin master`	subir 1ª vez y enlazar rama
`git push`	subir (rama ya enlazada)
`git pull`	bajar **y mezclar** (fetch + merge)
`git fetch origin`	bajar **sin mezclar** → `origin/master`
`git log origin/master`	ver lo bajado con fetch
`git merge origin/master`	mezclarlo después del fetch
`git branch -vv`	qué rama local va con qué remota

💡 Token (contraseña de GitHub)

GitHub → foto → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → Generate. Marcar **repo** y **workflow**. Se ve una sola vez.
En el push: **Username** = tu usuario · **Password** = el token.

⚠️ Antes de push/pull

Haz commit de lo pendiente. Crea el repo en GitHub **vacío** (sin README, .gitignore ni licencia).

⚡ Que no lo pida más

```
git config --global credential.helper store
```

5 · Ramas y merge

Comando	Hace
`git branch`	listar (* = actual)
`git branch --show-current`	rama actual
`git branch r`	crear (NO te mueve)
`git switch r` · `git checkout r`	moverse
`git switch -c r` · `git checkout -b r`	crear + moverse
`git checkout -b r origen`	crear desde otra rama/ tag
`git branch -a` · `-r`	todas · remotas
`git branch -m nuevo`	renombrar actual
`git branch -d r`	borrar (fusionada)
`git branch -D r`	borrar a la fuerza
`git push -u origin r`	subir rama
`git push origin --delete r`	borrar en GitHub
`git checkout r` (remota)	traer rama remota tras clonar

Merge	Hace
`git merge r`	trae `r` a la rama **donde estás**
`git merge --no-ff r`	siempre crea commit de merge
`git merge --abort`	cancelar merge con conflicto

💡 **Fast-forward vs 3 vías**

Fast-forward: la rama receptora no avanzó → solo mueve el puntero, sin commit nuevo.
3 vías: las dos avanzaron → crea "Merge branch 'x'" y abre el editor.

🎯 **Trampas**

· `git branch x` no te mueve: el siguiente commit va a la rama antigua.
· Antes de merge: estar en la rama que **recibe** y con status limpio.
· Al cambiar de rama, los ficheros de la otra "desaparecen" (viven allí).

Conflicto

```
<<<<<<< HEAD
<h1>Hola</h1>           ← tu rama
=======
<h1>Bienvenido</h1>     ← la otra
>>>>>>> titulo                  → editar, quitar marcas, git add, git commit (receta R5)
```

6 Tags

Comando	Hace
`git tag`	listar
`git tag -l 'v1.*'`	buscar por patrón
`git tag v1.0`	ligero (solo nombre)
`git tag -a v1.0 -m "M"`	anotado (autor, fecha, mensaje)
`git tag -a v0.9 -m "M" HASH`	etiquetado tardío
`git show v1.0`	ver tag
`git tag -d v1.0`	borrar local
`git push origin v1.0`	subir uno
`git push origin --tags`	subir todos
`git push origin --delete v1.0`	borrar en GitHub
`git push origin :refs/tags/v1.0`	borrar en GitHub (otra forma)
`git checkout -b r v1.0`	rama desde un tag

🎯 Cae fijo
`git push` normal **NO** sube los tags.

💡 Rama vs tag
Rama = marcapáginas que **avanza**. Tag = post-it **fijo** en un commit (versiones).

⚡ Hash de un commit viejo
`git log --oneline` → copia los 7 caracteres → `git tag -a nombre -m "M" HASH`

7 · GitHub Pages, fork y PR

GitHub Pages

```
git switch -c gh-pages
git add index.html style.css
git commit -m "Publica web"
git push -u origin gh-pages
```

Repo → **Settings → Pages** → Source: **Deploy from a branch** → `gh-pages` / `(root)` → **Save**.
URL: `https://USUARIO.github.io/REPO/` (1-2 min).
Progreso: pestaña Actions ("pages build and deployment").

Temas (Jekyll) solo con `README.md`: `_config.yml` → `theme: jekyll-theme-cayman`.

Fork + upstream + PR

```
# 1. Botón Fork en GitHub · 2. clonar TU fork
git clone https://github.com/USUARIO/repo.git
git remote add upstream URL_ORIGINAL
git fetch upstream
git merge upstream/main       # remoto/rama
git switch -c mejora
# ...commits...
git push -u origin mejora
```

GitHub → **Pull requests → New** (o Contribute → Open PR) → **base** = repo/rama destino · **compare** = tu rama → Create → **Merge pull request**.

🎯 Buenas prácticas
Añadir solo lo tocado (`git add f`) · no subir código a medias (un commit = un cambio completo) · seguir metodología de ramas (Gitflow) · mensajes según reglas (sección 2).

8 Gitflow y teoría SCV

Rama	Para qué	Sale de	Se fusiona en	Nombre
master	producción, vive siempre	—	—	master
develop	próxima versión, vive siempre	master	—	develop
feature	funcionalidad nueva	develop	develop	feature/x
release	preparar versión (solo arreglos)	develop	develop y master (+tag)	release-*
hotfix	arreglo urgente producción	**master**	develop y master (+tag)	hotfix-*

git flow	Git normal
`git flow init`	`git init` + commit + `git checkout -b develop master`
`feature start X`	`git checkout -b feature/X develop`
`feature finish X`	`git checkout develop` · `git merge --no-ff feature/X` · `git branch -d feature/X`
`feature publish X`	`git push origin feature/X`
`release start 1.2.0`	`git checkout -b release/1.2.0 develop`
`release finish 1.2.0`	`git checkout master` · `merge --no-ff release/1.2.0` · `git tag -a 1.2.0` · `git checkout develop` · `merge --no-ff release/1.2.0` · `branch -d release/1.2.0`
`hotfix start 1.2.1`	`git checkout -b hotfix/1.2.1 master`
`hotfix finish 1.2.1`	igual que release finish

Instalar: `sudo apt install git-flow`. Se recomienda: equipo > 2 personas, metodologías ágiles, cambios frecuentes, proyecto complejo.

Tipo SCV	Cómo	Ejemplos
Local	copias en tu PC	RCS
Centralizado	un servidor; **punto único de fallo**	CVS, SVN
Distribuido	copia completa en cada equipo	Git, Mercurial, Bazaar

💡 Git
· Linus Torvalds, 2005 (núcleo de Linux)
· **instantáneas**, no diferencias
· casi todo **local** (sin internet)
· **integridad** con hash SHA-1 (40 caracteres)
· 3 estados: modificado, preparado, confirmado

💡 SVN vs CVS
commits atómicos · ramas más eficientes · mejor con binarios · envía solo incrementos.

9 GitHub Actions: workflows

.github/workflows/*.yml

```yaml
name: CI
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
env:
  APP: tienda
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - name: Varios comandos
        run: |
          echo "Hola $APP"
          ls -la
      - run: ls
        working-directory: ./temp
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "test"
  deploy:
    needs: [build, test]
    if: github.ref_name == 'main'
    runs-on: ubuntu-latest
    steps:
      - env:
          KEY: ${{ secrets.DEPLOY_KEY }}
        run: echo "Desplegando $APP"
```

Clave	Significa
`name`	nombre visible
`on`	evento (obligatorio)
`jobs.ID`	id inventado
`runs-on`	máquina (obligatorio)
`steps`	pasos en orden, con **-**
`uses`	action `usuario/repo@v`
`with`	parámetros de la action
`run` / `run: |`	comando / varios
`needs`	esperar a otro job
`if`	condición
`env`	variables
`id` (step)	para leer sus outputs
`timeout-minutes`	límite (def. 360)

⚠️ **YAML**

2 espacios, nunca Tab · espacio tras `:` · versiones entre comillas: `'3.10'` · sin `checkout` no hay código.

💡 **Orden**

Jobs en **paralelo** salvo `needs`. Steps en **orden**. Si un job falla, los que lo necesitan se saltan (salvo `if: always()`). Cada job = runner nuevo y limpio.

⚡ **Ver resultado**

Pestaña **Actions** → ejecución → job → step. Manual: **Run workflow** (necesita `workflow_dispatch`).

on: eventos y filtros

```yaml
on: push
on: [push, pull_request]
on:
  push:
    branches: [ main, 'releases/*', '!releases/**-alpha' ]
    tags: [ 'v*' ]
    paths: [ '**.py' ]
  schedule:
    - cron: '0 8 * * 1'    # lunes 8:00 UTC
```

Filtro	Hace
`branches` / `tags`	solo esas (`!` excluye)
`branches-ignore`	todas menos esas
`paths` / `paths-ignore`	según ficheros cambiados
`types`	opened, edited...

cron	min hora día mes díaSem
`'*/15 * * * *'`	cada 15 min
`'30 7 * * *'`	cada día 7:30
`'0 18 * * 5'`	viernes 18:00 (0=dom)

Contextos ${{ }}, env, secrets

Expresión	Vale
`github.ref_name`	rama/tag
`github.event_name`	push, pull_request...
`github.actor`	usuario
`github.sha`	hash commit
`runner.os`	Linux/Windows/ macOS
`matrix.X`	valor de la matriz
`secrets.X` · `secrets.GITHUB_TOKEN`	secreto · automático
`steps.ID.outputs.X`	output de un step
`inputs.X`	input (custom action)

💡 env en 3 niveles

workflow → job → step. Si se repite, gana **la más concreta (step)**. Ya existen: `GITHUB_SHA`, `GITHUB_REF`, `GITHUB_EVENT_NAME`, `HOME` ...

⚡ Secret

Repo → **Settings → Secrets and variables → Actions → New repository secret**. En el log sale `***`.

💡 Condiciones

`if: always()` · `success()` · `failure()` · `github.ref_name != 'main'` · `A && B || C` = si A → B, si no → C

matrix y artifacts

```yaml
strategy:
  fail-fast: false     # def. true: cancela el resto si uno falla
  matrix:
    os: [ubuntu-latest, windows-latest]
    python: ['3.11', '3.12', '3.13']
    include:
      - os: macos-latest
        python: '3.12'
runs-on: ${{ matrix.os }}
# 2 × 3 = 6 jobs (+1 del include = 7)
```

```yaml
- uses: actions/upload-artifact@v4
  with:
    name: resultado
    path: salida.csv
# otro job: actions/download-artifact@v4
# caché deps: actions/cache (key, path, restore-keys)
# artifacts duran 90 días
```

Teoría Actions y CI/CD

Pieza	Qué es
Workflow	fichero YAML; un evento lo dispara; 1 fichero = 1 workflow
Evento	push, PR, cron, manual...
Job	steps en el mismo runner; paralelos
Step	tarea: `uses` o `run`; en orden
Action	pieza reutilizable más pequeña
Runner	máquina que ejecuta jobs

	GitHub-hosted	Self-hosted
Dónde	nube GitHub	tu máquina
Mantenim.	ninguno	tú
Coste	gratis 2000 min/ mes	sin pagar uso
Ventaja	limpio, listo	red local, hardware a medida
runs-on	`ubuntu-latest`	`[self-hosted, linux]`

Jerarquía: Workflow → Job → Step → Action

CI · Integración continua	CD · Entrega continua	CD · Despliegue continuo
Integrar varias veces al día con tests automáticos	Siempre listo; el despliegue lo aprieta **una persona**	Pasa tests → producción **automático**

Alternativas: GitLab CI/CD, Jenkins, Travis CI.

10 Práctica euro.py

repo titansmc/actions · rama main

#	Pide	Cómo
1	Clonar el repo	`git clone https://github.com/titansmc/actions.git`
2	Repo `actions` en tu GitHub	crearlo vacío → `git remote set-url origin URL` → `git push -u origin main`
3	`pip install -r requirements.txt`	step `run: pip install -r requirements.txt` + fichero `requirements.txt` (un paquete por línea)
4	Rama `test` con Python 3.12	`git switch -c test` → `python-version: '3.12'` → **añadir `test` a branches** → `git push -u origin test`
5	Rama `development` solo USD	`git switch main` → `git switch -c development` → `run: python ./euro.py USD` → **añadir a branches** → push

YAML del profe (main):

```yaml
name: Monedes
on:
  push:
    branches: [ main ]
jobs:
  create-list:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout del repo
        uses: actions/checkout@v4
      - name: Configurem python
        uses: actions/setup-python@v7
        with:
          python-version: '3.12'
      - name: Preparem l'entorn amb els paquets
        run: pip install -r requirements.txt
      - name: Correm l'script
        run: python ./euro.py
```

💡 **euro.py**

sin args: todas · `USD GBP`: solo esas · `--csv f.csv` · `--data AAAA-MM-DD`. requirements: `requests`, `pandas`.

Versión pro (un YAML para las 3 ramas):

```yaml
on:
  push:
    branches: [ main, test, development ]
jobs:
  create-list:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ github.ref_name == 'test' && '3.12' || '3.11' }}
      - run: pip install -r requirements.txt
      - if: github.ref_name != 'development'
        run: python ./euro.py
      - if: github.ref_name == 'development'
        run: python ./euro.py USD
```

🎯 **Clave**

Cada rama tiene **su propia copia** del YAML. Si `branches` no incluye tu rama, **no se ejecuta**.

⚡ **Probar en local (Ubuntu 24)**

`python3 -m venv venv` · `source venv/bin/activate` · `pip install -r requirements.txt` · `python euro.py USD` · `venv/` al .gitignore

11 Custom Actions

```yaml
# action.yml
name: 'Hello World'            # OBLIGATORIO
description: 'Saluda'           # OBLIGATORIO
author: 'Fabián'
inputs:
  who-to-greet:
    description: 'A quién'
    required: true
    default: 'World'          # único opcional
outputs:
  time:
    description: 'Hora'
runs:                          # OBLIGATORIO
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
branding:
  icon: 'award'
  color: 'green'
```

```
# Dockerfile
FROM alpine:3.10
COPY entrypoint.sh /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

```
# entrypoint.sh   (+ chmod +x entrypoint.sh)
#!/bin/sh -l
echo "Hello $1"
echo "time=$(date)" >> $GITHUB_OUTPUT
# apuntes (obsoleto): echo "::set-output
name=time::$time"
```

```yaml
# usarla en un workflow
steps:
  - uses: actions/checkout@v4    # OBLIGATORIO
  - id: hello
    uses: ./                      # o ./.github/
actions/x
    with:
      who-to-greet: 'Mona'
  - run: echo "${{ steps.hello.outputs.time }}"
```

	🐳 Docker	⚡ JavaScript
Velocidad	lenta	rápida
Runner	**solo Linux**	todos
Dependencias	en el contenedor	empaquetar
Ventaja	consistente	código simple

Tercer tipo: **composite** (varios steps en una action).

runs JS	runs Docker
`using: 'node20'` ✅ `main: 'index.js'` ✅ `pre` / `pre-if` `post` / `post-if`	`using: 'docker'` ✅ `image` ✅ `args` · `entrypoint` `pre-entrypoint` · `post-entrypoint`

> 🎯 **Cae seguro**
> Obligatorios: **name, description, runs**. Input `who-to-greet` → variable `INPUT_WHO-TO-GREET` (mayúsculas, espacios → _).

```
# index.js
const core = require('@actions/core');
try {
  const n = core.getInput('who-to-greet');
  console.log(`Hello ${n}!`);
  core.setOutput('time', new
Date().toTimeString());
} catch (e) { core.setFailed(e.message); }
# npm init -y
# npm install @actions/core @actions/github
# npm i -g @vercel/ncc → ncc build index.js
#   → main: 'dist/index.js'
```

uses:	Origen
`usuario/repo@v1`	repo público
`usuario/repo/dir@v1`	subcarpeta
`./ruta`	tu repo
`docker://img:tag`	Docker Hub

12 Errores típicos → solución

Mensaje / síntoma	Solución
`Please tell me who you are`	`git config --global user.name "..."` y `user.email`
`nothing to commit` tras `touch`	El fichero ya existía igual. Cambia el contenido: `echo "x" >> f`
`Changes not staged` al hacer commit	Faltó `git add` (o usa `git commit -am` si ya estaba seguido)
`deleted: f` sin preparar	Borraste con `rm`: `git rm f` y commit
Comando raro de Linux (`reset`, `gs` ...)	Te faltó el `git` delante
`-- soft`, `-- hard` no hacen lo esperado	Sin espacio: `--soft`, `--hard`
Se abre vi y no sales	**Esc** → `:wq` → Enter. Para siempre: `git config --global core.editor nano`
`not a git repository`	No estás en la carpeta del repo: `cd` + `pwd`, o falta `git init`
`remote origin already exists`	`git remote set-url origin URL`
`src refspec master does not match any`	Rama se llama `main` o no hay commits: `git branch`, `git branch -m master`
`rejected ... fetch first` / `non-fast-forward`	GitHub tiene commits que tú no: `git pull` y luego `git push` (o creaste el repo con README)
`Authentication failed` / pide password	Usa el **token** como password, no tu contraseña
`refusing to allow a Personal Access Token to create or update workflow`	Token nuevo con permiso **workflow**
`branch is not fully merged`	Tiene commits sin fusionar: fusiona primero o `git branch -D`
`CONFLICT (content)`	Receta R5
`Your local changes would be overwritten` al cambiar de rama	Haz commit antes de `git switch`
Workflow no se ejecuta en mi rama	Falta la rama en `on: push: branches`
`Invalid workflow file`	Sangría (2 espacios, sin Tab) o falta espacio tras `:`
Step no encuentra ficheros	Falta `- uses: actions/checkout@v4`
Custom Docker action: `permission denied`	`chmod +x entrypoint.sh` y commit
`externally-managed-environment` (pip)	Usa venv: `python3 -m venv venv && source venv/bin/activate`
Tags no aparecen en GitHub	`git push origin --tags`

🎯 Checklist final antes de entregar

☐ `git status` limpio · ☐ en la rama que piden · ☐ `git log --oneline --graph --all --decorate` muestra lo pedido · ☐ ramas que piden borrar, borradas · ☐ tags creados **y subidos** · ☐ `git push` hecho · ☐ en GitHub: ficheros, ramas, tags y pestaña Actions en ✅ verde.